

Aprenentatge Automàtic

Estudi de la gravetat dels accidents de trànsit a Catalunya entre el 2010 i 2020

Curs 2025-2026

MARINA FERRÉ MARTINEZ 1632759

CLARA VALLS MORESO 1672780

MATHILDE BUFFARD 1775251

Índex

1	Introducció	3
1.1	Breu introducció al tema de l'aprenentatge automàtic.	3
1.2	Motivació i justificació de la tria del conjunt de dades.	3
1.3	Explicació dels objectius específics del treball	3
2	Descripció del conjunt de dades	4
2.1	Explicació de l'origen del conjunt de dades.	4
2.2	Descripció de les variables incloses.	4
2.3	Justificació de la rellevància del conjunt de dades per al projecte.	5
3	Exploració de les dades	6
3.1	Estadístiques descriptives de les variables principals	6
3.2	Visualització gràfica de les dades	6
4	Aplicació d'algorismes d'aprenentatge automàtic	9
4.1	Preprocessament i Preparació de les Dades	9
4.2	Mètrica de selecció: AUC-ROC i selecció d'hiperparàmetres amb cross-validation	10
4.3	Màquines de Vector de Suport (SVM)	10
4.3.1	Optimització de l'Hiperparàmetre C	10
4.3.2	Nucli Lineal	10
4.3.3	Nucli Polinòmic	11
4.3.4	Nucli Radial	11
4.4	knn	11
4.4.1	Selecció de la k i Evitació d'Empats	11
4.5	Decision Trees	11
5	Resultats i interpretació	12
5.1	Presentació dels resultats obtinguts amb cadascun dels models.	12
5.2	Comparació dels models en termes de rendiment.	12
5.2.1	Decision Tree (Gini)	12
5.2.2	Decision Tree (Entropy)	12
5.2.3	SVM Polinòmic	13
5.2.4	SVM RBF (Radial Basis Function)	13
5.2.5	SVM Lineal	13
5.2.6	k-Nearest Neighbors (k-NN)	13
5.3	Interpretació dels resultats i implicacions dels resultats en el context del conjunt de dades utilitzat.	14
6	Conclusions i Treball Futur	15
7	Annexos	16

1 Introducció

1.1 Breu introducció al tema de l'aprenentatge automàtic.

La Intel·ligència Artificial (IA) engloba tots els algorismes que intenten que una màquina imiti les funcions cognitives dels humans, com l'aprenentatge o la resolució de problemes. L'Aprenentatge Automàtic (Machine Learning) és el subcamp de la IA que utilitza les matemàtiques i l'estadística per a fer algorismes que permeten: analitzar dades, construir models que milloren (prediuen millor) a mesura que es van alimentant de més dades, fer prediccions amb aquests models, que ajuden a prendre decisions i a generar coneixement.

1.2 Motivació i justificació de la tria del conjunt de dades.

La tria d'aquest conjunt de dades respon a una inquietud que ens toca de molt a prop. Com a usuaris setmanals de la línia d'autobús que connecta les Terres de l'Ebre (Tortosa i Amposta) amb Barcelona, som testimonis recurrents de la realitat viària. En aquests trajectes llargs, sovint ens convertim en espectadores d'accidents o de les retencions que generen, fet que ens ha portat a reflexionar sobre la seguretat a les nostres carreteres.

Més enllà de l'experiència personal, som conscients que és una problemàtica d'impacte social majúscul. Per això, hem volgut passar de l'observació a l'acció: utilitzar aquestes dades no només per entendre el passat, sinó per construir un model capaç de predir la gravetat d'un accident en funció de variables clau.

1.3 Explicació dels objectius específics del treball

L'objectiu principal d'aquest treball és aplicar tècniques d'aprenentatge automàtic que hem estudiat a un cas real i complex com és la sinistralitat a les carreteres catalanes. La finalitat és entrenar models capaços de detectar patrons en els accidents greus. Per fer-ho, hem estructurat el projecte en cinc fases que defineixen els nostres objectius específics:

1. Descripció del conjunt de dades Abans de començar a treballar, el primer pas és entendre el conjunt de dades. Aquest objectiu se centra a documentar l'origen de les dades i l'autoria, així com analitzar l'estructura de les variables disponibles. Justificarem per què aquest dataset és rellevant per al nostre estudi, assegurant-nos que tenim una base sòlida per construir els models.

2. Exploració de les dades En aquesta fase, realitzarem una anàlisi exploratòria exhaustiva utilitzant estadística descriptiva i visualitzacions gràfiques (com histogrames). La intenció és detectar a simple vista tendències, valors atípics o possibles biaixos que puguin distorsionar els resultats posteriors, permetent-nos netejar i preparar millor la informació.

3. Aplicació dels algorismes d'aprenentatge automàtic L'objectiu és implementar i ajustar tres algorismes d'aprenentatge supervisat: *Support Vector Machines* (SVM), *k-Nearest Neighbors* (k-NN) i Arbres de Decisió (DT). Justificarem per què són adequats per a aquest context i els entrenarem correctament per evitar problemes com el sobreajustament.

4. Avaluació i interpretació dels resultats Avaluarem el rendiment de cada algorisme mitjançant mètriques i els compararem entre ells. Busquem interpretar què signifiquen aquests resultats en el context de la seguretat viària: quins factors tenen més pes en la gravetat d'un accident?

5. Conclusions i propostes de futur Finalment, l'objectiu és extreure conclusions generals sobre la utilitat del *Machine Learning* en aquest camp, però també fer autocrítica. Identificarem les limitacions del nostre treball i proposarem possibles millores o línies d'investigació futures que podrien perfeccionar els models.

2 Descripció del conjunt de dades

2.1 Explicació de l'origen del conjunt de dades.

Per a aquest treball hem escollit el conjunt de dades dels **accidents de trànsit a Catalunya entre els anys 2010 i 2020** ¹. Les dades s'han obtingut del lloc web *Kaggle*; han estat publicades per Jordi Garcia Castillon, però el seu origen és el Departament d'Interior, Servei Català de Trànsit ².

Aquest conjunt de dades s'ha descarregat en format `.csv` (Comma-Separated Values), és a dir, un document de text amb els camps separats per comes. Posteriorment, el fitxer s'ha llegit en *Python* mitjançant la llibreria *pandas*, convertint-lo en un *data frame*. D'aquesta manera, s'ha obtingut una base de dades estructurada en la qual cada fila correspon a un accident ocorregut i cada columna representa una variable amb informació associada a aquest accident.

2.2 Descripció de les variables incloses.

El *data frame* original contenia 58 columnes, és a dir, 58 variables, i 16,8 mil files, corresponents a accidents de trànsit. D'entre les 58 variables disponibles al conjunt de dades, hem construït una nova variable objectiu (*target*) per al nostre model de classificació, anomenada **Morts_Binaria**. Aquesta variable dependent té com a finalitat distingir els accidents amb conseqüències greus. Així doncs, hem definit la classe negativa (0) si no hi ha morts i la classe positiva (1) si n'hi ha.

D'aquesta manera, el model s'entrenarà per detectar quins factors converteixen un accident en un accident greu.

De les 58 variables inicials del conjunt de dades, se n'han seleccionat 9. Aquesta selecció s'ha realitzat considerant que aquestes variables són les més rellevants per a la predicció del nombre de víctimes. A continuació es descriu breument cadascuna d'elles: **Any**: any en què es va produir l'accident, **dat**: data del dia en què va tenir lloc l'accident, **D_TIPUS_VIA**: tipus de via. Hi ha sis categories: via urbana (inclou carrer i carrer residencial), carretera convencional, autovia, autopista, camí rural/pista forestal i altres, **nomCom**: nom de la comarca on es va produir l'accident, **F_UNITATS_IMPLICADES**: nombre d'unitats implicades en l'accident, com ara vianants, motocicletes, ciclomotors, camions o turismes, **C_VELOCITAT_VIA**: velocitat màxima permesa a la via, **D_SENTITS_VIA**: nombre de sentits de circulació de la via, que pot ser un sentit, doble sentit o desconegut, **grupDiaLab**: indica si l'accident s'ha produït en un dia feiner o en cap de setmana, **hor**: hora

en què es va produir l'accident.

Tanmateix, aquestes variables, en el seu format original, no són òptimes per al seu tractament, de manera que ha estat necessari dur a terme un procés de neteja i redefinició:

La variable **dat** s'ha modificat i s'ha transformat en una nova variable anomenada **mes**, amb valors enters entre 1 i 12. Aquesta transformació es justifica perquè la informació de l'any ja està recollida a la variable **Any**, i el dia exacte no s'ha considerat necessari, ja que la variable **grupDiaLab** aporta informació suficient sobre el tipus de dia, evitant així la duplicació d'informació.

La variable **D_SENTITS_VIA** s'ha recodificat passant de les categories *un sentit*, *dobles sentits* i *desconegut* als valors 1, 2 i NaN, respectivament. Posteriorment, s'han eliminat les observacions amb el sentit de la via desconegut.

La variable **grupDiaLab** s'ha convertit de categòrica a binària, assignant el valor 1 als dies feiners i el valor 0 als caps de setmana.

Finalment, la variable **hor**, inicialment expressada en format hores:minuts, s'ha transformat en dues noves variables: **hora_sin** i **hora_cos**. Per fer-ho, primer s'ha expressat l'hora com el nombre de minuts transcorreguts des de l'inici del dia. Considerant que les 23:59 corresponen a 1439 minuts, s'han definit les transformacions:

$$\text{hora_sin} = \sin\left(2\pi \cdot \frac{\text{minuts_dia}}{1440}\right), \quad \text{hora_cos} = \cos\left(2\pi \cdot \frac{\text{minuts_dia}}{1440}\right).$$

D'aquesta manera, una variable discreta com l'hora s'ha convertit en una representació contínua i cíclica, garantint que la distància entre les hores 23:59 i 00:00 sigui mínima.

Així doncs, en aquest estudi la variable objectiu és **Morts_Binaria**. Pel que fa a les variables explicatives (*features*), la classificació final és la següent:

- **Variable Objectiu (Target):** Morts_Binaria (Binària).
- **Variables Categòriques Nominals:** D_TIPUS_VIA i nomCom.
- **Variables Binàries:** D_SENTITS_VIA i grupDiaLab.
- **Variables Numèriques Discretes:** Any, C_VELOCITAT_VIA, F_UNITATS_IMPLICADES i Mes.
- **Variables Numèriques Contínues:** hora_sin i hora_cos.

2.3 Justificació de la rellevància del conjunt de dades per al projecte.

Aquest conjunt de dades el trobem idoni per a aquest treball. Tenim una base de dades molt completa, amb moltes dades i molt fiable perquè fou publicada directament per la Generalitat. A més, amb la nostra tria de dades hem mantingut aquesta riquesa de les dades, però eliminant aquelles variables que ens aportaven informació duplicada, i aquelles que hem considerat que no serien tan rellevants, així acabant tenint unes variables molt riques però en de dimensió reduïda per poder aplicar còmodament els models d'aprenentatge automàtic.

3 Exploració de les dades

3.1 Estadístiques descriptives de les variables principals

L'aplicació de l'anàlisi descriptiva agrupada per la variable binària de mortalitat (*Morts_Binaria*) revela patrons diferencials clau entre els accidents que resulten en víctimes mortals (Grup 1) i aquells que no (Grup 0). L'objectiu d'aquesta segmentació és identificar potencials factors de risc o de protecció associats a la fatalitat.

La mitjana de la velocitat de la via en el Grup 0 (accidents sense morts) és de 84.55 km/h, no molt llunyana als 87.92 km/h observats en el Grup 1 (accidents amb morts). I amb una petita dispersió de 25.00 per al Grup 0 i 21.30 per al Grup 1. Notem que aquesta variable no té soroll gràcies a que hem eliminat els valors atípics més grans de 120 km/h ja que no hi ha carreteres amb més d'aquesta velocitat.

Els resultats indiquen que el nombre d'unitats implicades no és un factor fortament discriminant respecte a la mortalitat. La mitjana d'unitats en el Grup 0 és d'1.91, mentre que en el Grup 1 és marginalment superior, d'1.95. En ambdós casos, el 75% dels accidents (Q3) no superen les dues unitats implicades, confirmant que la majoria d'accidents, tant fatals com no fatals, són col·lisions d'un vehicle sol o de dos unitats.

3.2 Visualització gràfica de les dades

L'anàlisi visual de la distribució de sinistres, segmentada per la variable objectiu *Morts_Binaria*: 0 = Sense Morts, 1 = Amb Morts, complementa l'anàlisi descriptiva i permet identificar de forma immediata la relació entre les variables explicatives i la fatalitat de l'accident. La Figura 1 mostra la comparació entre dies feiners i caps de setmana:

Absència de Morts (Grup 0): El recompte de sinistres sense morts és notablement superior en dies feiners que en caps de setmana.

Presència de Morts (Grup 1): El nombre d'accidents mortals és gairebé el doble en dies feiners que en caps de setmana.

Atès que el trànsit és generalment més elevat i diversificat durant els dies feiners (trànsit laboral, comercial, etc.), la major freqüència absoluta d'accidents mortals en aquests dies sembla lògica.

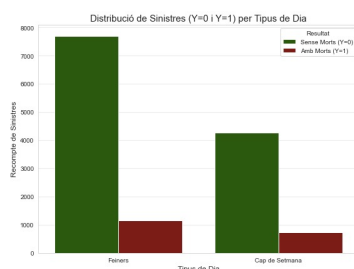


Figura 1: Dia Laborable vs Cap de Setmana

La Figura 2 mostra la gravetat en funció dels sentits de circulació:

Doble Sentit: Aquest tipus de via acumula la majoria d'accidents, tant sense morts com amb morts. La gran majoria de sinistres mortals es produeixen en vies de doble sentit.

Sentit Únic: El nombre d'accidents, i especialment el de mortals, és molt inferior.

Aquest resultat és fortament predictiu, ja que la configuració de doble sentit incrementa el risc de col·lisions frontals o laterals, que són intrínsecament més greus.

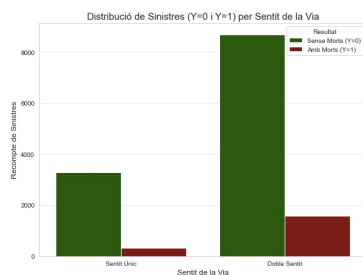


Figura 2: Sentit de la Via

La Figura 3 il·lustra la freqüència dels sinistres per tipus de via:

Vies més Freqüents: Les vies urbanes (carrer i carrer residencial) i les carreteres convencionals aglutinen la majoria d'accidents, tant mortals com no mortals.

Vies amb Més Mortalitat (en valor absolut):

Les Carreteres Convencionals presenten el nombre més alt d'accidents amb morts (Grup 1).

Les Vies Urbanes presenten un alt nombre d'accidents totals, però una menor proporció de mortalitat respecte a les convencionals.

La sinistralitat de les carreteres convencionals (generalment amb doble sentit i sense separació física) es confirma com el focus principal de la mortalitat viària.

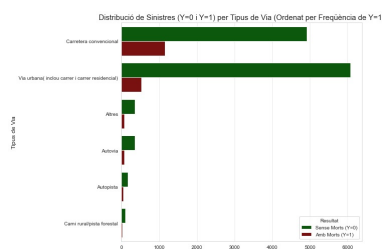


Figura 3: Tipus de Via

La Figura 4 mostra el recompte de sinistres per comarca (Top 10 comarques amb més accidents).

Les comarques amb major densitat de població i xarxa viària (com el Barcelonès, el Vallès Oriental, i el Maresme) acumulen el major nombre de sinistres totals.

El Barcelonès destaca com a comarca amb un alt nombre d'accidents mortals.

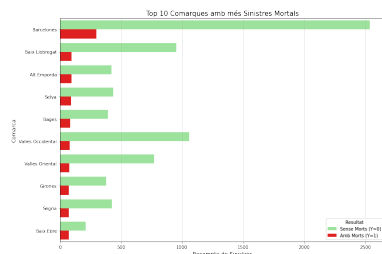


Figura 4: Top 10 Comarques amb més accidents

La Figura 5 mostra l'evolució del nombre d'accidents al llarg dels anys (2010 a 2020) : S'observa una tendència decreixent en el nombre total d'accidents al llarg del període analitzat, tant per al Grup 0 (Sense Morts) com per al Grup 1 (Amb Morts). Ara bé, la proporció d'accidents mortals ($Y = 1$) es manté relativament estable respecte al total d'accidents cada any.

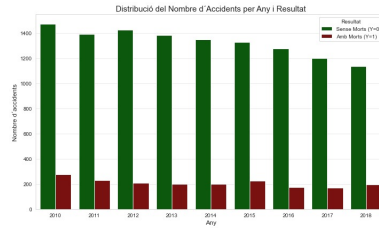


Figura 5: Distribució del nombre d'accidents per any

La Figura 6 il·lustra la freqüència de sinistres segons el mes de l'any. El nombre més alt d'accidents totals es produeix durant els mesos d'estiu, amb el pic al mes de juliol (7). Tot i que la majoria d'accidents mortals es produeixen als mesos de màxima sinistralitat (estiu), no s'observa un patró clar que indiqui que un mes concret sigui desproporcionadament més perillós en termes de fatalitat.

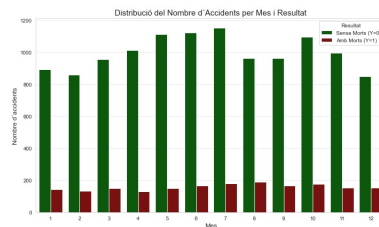


Figura 6: Distribució del nombre d'accidents per mes

La Figura 7 confirma l'observació de l'anàlisi descriptiva:

La immensa majoria d'accidents, tant amb morts com sense, impliquen 1 o 2 unitats. El nombre d'unitats no és un gran factor diferenciador de la mortalitat.

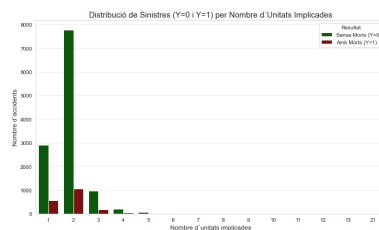


Figura 7: Nombre d'unitats implicades en els accidents

La Figura 8 il·lustra la freqüència d'accidents per a cada límit de velocitat. La majoria d'accidents, tant mortals com no mortals, es produeixen en límits de velocitat estàndard, principalment a 100 km/h (associat a accidents sense morts) i 80 km/h. Aquesta concentració en els límits de 80 i 100 km/h concorda amb la major sinistralitat en les Carreteres Convencionals (Figura 3). Pel que fa als accidents mortals, els punts amb major fatalitat es troben en els límits de 80 km/h i 100 km/h, subratllant que la majoria d'accidents greus es produeixen en vies ràpides i interurbanes.

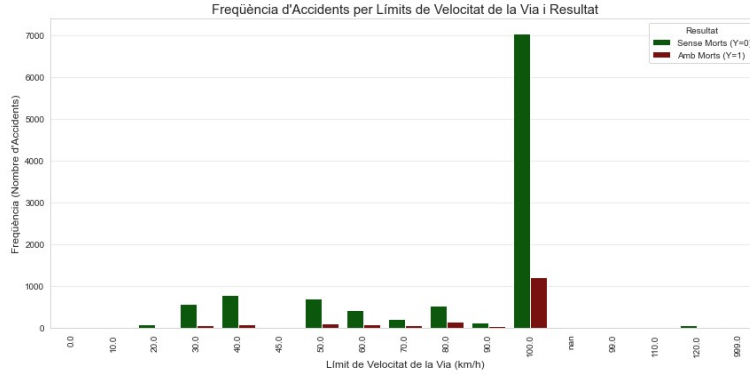


Figura 8: Freqüència d'Accidents per tots els Límits de Velocitat de la Via

La Figura 9 presenta la distribució horària amb un histograma. Hi ha un clar pic d'accidents tan mortals com no mortals entre les 00:00 i les 03:00, intuïtivament no sembla lògic ja que és just quan tothom dorm, però potser també és quan hi ha menys velocitat i la gent pot anar més cansada, borratxa o drogada.

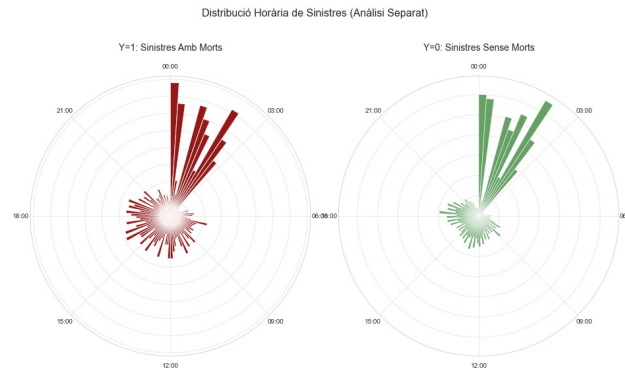


Figura 9: Freqüència d'Accidents al Llarg del Dia

4 Aplicació d'algorismes d'aprenentatge automàtic

En aquesta secció detallarem la metodologia seguida per a la implementació i l'entrenament dels models predictius. L'objectiu és classificar la variable `Morts_Binaria` (on 0 indica un accident sense víctimes mortals i 1 un accident amb víctimes mortals). Per a fer-ho, hem utilitzat els algorismes: *Support Vector Machines* (SVM), *k-Nearest Neighbours* (k-NN) i *Decision Trees* (DT) .

4.1 Preprocessament i Preparació de les Dades

Abans d'entrenar els models, hem hagut d'estandarditzar les variables numèriques. Aquest pas és vital en models com l'SVM i el k-NN, ja que es basen en el càlcul de distàncies. En el nostre dataset, les variables tenen magnituds molt diferents: l'Any ronda els 2000, la velocitat arriba a 120, però les variables transformades com `hora_sin` fluctuen només entre -1 i 1.

4.2 Mètrica de selecció: AUC-ROC i selecció d'hiperparàmetres amb cross-validation

Un punt clau del treball és la tria de la mètrica per optimitzar els hiperparàmetres (com la C en SVM o la k en k-NN). Hem descartat l'accuracy (exactitud) perquè, en tenir només un 15 % d'accidents mortals, el model podria obtenir un 85 % d'èxit simplement dient que ningú mor. En canvi, hem triat l'AUC-ROC. Aquesta mètrica considera com evoluciona l'equilibri entre el TPR i el FPR en tots els escenaris possibles. D'aquesta manera, l'optimització busca el model que més s'apropi al punt ideal (0,1), on detectem tots els positius sense cometre cap fals positiu, sent així molt més robusta davant el desequilibri de dades que presenta el nostre conjunt de dades.

Per garantir que els nostres models siguin robustos i tinguin una bona capacitat de generalització, hem implementat una estratègia de validació creuada de 5 particions (k-fold Cross-Validation, $cv = 5$) en totes les fases d'ajust d'hiperparàmetres. Aquest enfocament ens permet assegurar que els resultats no depenguin d'una divisió arbitrària de les dades.

Tots els cops que fem un `train_test_split`, les dades es barregen i es divideixen a l'atzar, és per això que com la nostra variable target està desequilibrada, hem afegit la comanda `stratify=y`, així assurem que la proporció de classes es manté tant al conjunt d'entrenament (train) com al de prova (test).

4.3 Màquines de Vector de Suport (SVM)

El funcionament de l'SVM es basa intrínsecament en el càlcul de distàncies euclidianes. Per aquest motiu, l'ús de variables categòriques mitjançant codificació numèrica simple (Label Encoding) resulta inadequat. Si assignéssim valors enters correlatius a categories nominals (com ara l'1 i el 2), l'algorisme interpretaria una relació d'ordre o de proximitat que no existeix en la realitat, distorsionant completament la geometria del model. Per tal d'evitar aquest biaix, hem optat per treballar exclusivament amb variables numèriques. Per a aquest model, s'ha utilitzat l'`StandardScaler` per centrar les variables en 0 amb una desviació estàndard d'1.

4.3.1 Optimització de l'Hiperparàmetre C

Per a cada tipus de nucli, s'ha realitzat una cerca en graella (*GridSearchCV*) provant els valors $C \in \{0.1, 1, 10, 100\}$. El paràmetre C actua com un regulador del compromís entre l'error de classificació i l'amplada del marge:

- Un valor de C **baix** prioritza un marge més ample, essent més permissiu amb els errors en les dades d'entrenament per afavorir la generalització.
- Un valor de C **alt** penalitza severament cada error de classificació, forçant el model a ajustar-se estrictament a les dades, amb el risc de patir *overfitting*.

4.3.2 Nucli Lineal

Aquest nucli busca una separació mitjançant un hiperplà recte. Si el rendiment d'aquest nucli és baix, és un indicador directe que la mortalitat dels accidents no és linealment separable respecte a les variables estudiades, fet que justificaria l'ús de mètodes més complexos.

4.3.3 Nucli Polinòmic

S'ha explorat la complexitat de les relacions mitjançant nuclis de grau 2 (quadràtic) i grau 3 (cúbic). El nucli polinòmic permet generar fronteres de decisió no lineals a partir de les interaccions entre les variables originals. És especialment eficaç en aquest dataset on la mortalitat pot dependre d'una combinació específica.

4.3.4 Nucli Radial

El nucli de base radial és capaç de modelar relacions extremadament complexes creant "regions" d'influència al voltant dels punts. En aquest cas, s'ha optimitzat tant la C com el paràmetre γ (**gamma**):

- γ **alt**: Defineix una influència local, on només els punts molt propers a la frontera de decisió tenen pes (fronteres molt ajustades).
- γ **baix**: Defineix una influència més global, creant fronteres més suaus i arrodonides.

4.4 knn

En aquest cas hem treballat amb totes les dades i, com Scikit-learn no té la distància de Gower, hem optat pel One-Hot Encoding per a les variables categòriques. Aquesta tècnica ens ha permès transformar les comarques i els tipus de via en dimensions independents de 0 o 1, eliminant qualsevol jerarquia i situant totes les categories a la mateixa distància conceptual per al model. A continuació, hem aplicat el MinMaxScaler, ja que el k-NN es basa exclusivament en la proximitat geomètrica. Sense aquest escalat al rang $[0, 1]$, variables com la velocitat (fins a 120 km/h) haurien dominat injustament sobre les variables binàries; amb la normalització, hem garantit que tots els factors influeixin de manera equitativa en el càlcul de la distància. Finalment, mitjançant el GridSearchCV, hem optimitzat el nombre de veïns (k) i hem comparat la distància de Manhattan ($p = 1$), amb l'Euclidiana ($p = 2$). Aquesta comparativa ha estat clau per adaptar el model a la dispersió real dels accidents a Catalunya, assegurant que la predicció de mortalitat es basi en una semblança geomètrica real entre els sinistres i no només en la repetició de casos.

4.4.1 Selecció de la k i Evitació d'Empats

En la configuració del *GridSearchCV* per al k-NN, s'han testejat valors de k compresos entre 1 i 25. Hem testejat únicament **números senars** per al paràmetre k . L'objectiu d'utilitzar valors senars és evitar situacions d'empat en la votació.

4.5 Decision Trees

A diferència dels models anteriors, per als Decision Trees no ha estat necessari estandarditzar les dades. Això es deu al fet que aquest algorisme no calcula distàncies, sinó que realitza particions lògiques (regles si/aleshores) sobre les variables originals. Aquesta característica el fa especialment robust davant de dades amb diferents escales o valors atípics (outliers). Hem fet dos tipus d'arbres amb criteri Gini i amb criteri Entropy

En la fase d'entrenament, hem utilitzat GridSearchCV per optimitzar la profunditat màxima de l'arbre (`max_depth`), ja que si la profunditat és massa baixa, l'arbre és massa simple i pot fer underfitting i si és massa alta pot sobreajustar-se a les dades (overfitting).

L'ús d'arbres ens ha permès extreure el mètode d'importància de les variables, identificant quins factors són els predictors més potents del risc de mort, les variables situades a la part superior de l'arbre (més a prop de l'arrel) són les més informatives.

5 Resultats i interpretació

5.1 Presentació dels resultats obtinguts amb cadascun dels models.

Taula 1: Comparativa Final de Models

Model	accuracy	recall_min	f1_classmin	f1_classmaj	bal_acc	mcc	roc_auc
Decision Tree (Gini)	0.622667	0.647388	0.332216	0.737041	0.632932	0.189651	0.632932
Decision Tree (Entropy)	0.582635	0.694030	0.325317	0.697866	0.628888	0.181770	0.628888
SVM Polinòmic	0.428996	0.750000	0.275815	0.528689	0.562283	0.091566	0.562283
SVM RBF	0.535840	0.583955	0.267293	0.660333	0.555818	0.078631	0.555818
SVM Lineal	0.341628	0.822761	0.265983	0.403139	0.541403	0.067507	0.541403
k-Nearest Neighbors	0.853665	0.003731	0.007339	0.921010	0.500758	0.010837	0.500758

5.2 Comparació dels models en termes de rendiment.

5.2.1 Decision Tree (Gini)

Hem obtingut una `max_depth=4`. Notem que un arbre de profunditat 4 és molt fàcil de visualitzar i explicar. Aquest model presenta el millor equilibri general entre la capacitat de detectar la classe minoritària i mantenir una precisió global decent.

Amb un **Recall_min** de 0.65, és capaç d'identificar correctament el 65% dels casos de la classe minoritària. Tot i que la seva **Accuracy** (0.62) pot semblar modesta, la **Balanced Accuracy** (0.63) i el **ROC AUC** (0.63) indiquen que el model discrimina millor que l'atzar i de manera més estable que la resta.

El seu **MCC** (0.19) és el més alt de la taula. Com que el coeficient de correlació de Matthews és una de les mètriques més fiables en classes desequilibrades, això suggereix que aquest és el model més robust tècnicament. La diferència entre `f1_classmaj` (0.74) i `f1_classmin` (0.33) mostra que encara li costa precisar la classe petita (generant falsos positius), però és el resultat més útilitzable".

5.2.2 Decision Tree (Entropy)

S'ha seleccionat una `max_depth=5`, tot i que la que optimitza és 8, el auc és molt similar i la dificultat no compensa.

Comparat amb el criteri Gini, aquest model sacrifica **Accuracy** global (0.58 vs 0.62) a canvi d'un major **Recall_min** (0.69 vs 0.65). Això significa que és més sensible detectant la classe crítica, però ho paga generant més falsos positius, com reflecteix la baixada en l'`f1_classmaj`. La seva **Balanced Accuracy** i **ROC AUC** (0.63) són gairebé idèntiques al Gini. Per tant, si la prioritat absoluta és trobar el màxim de positius reals encara que hi hagi més "falses alarmes", aquest model seria preferible al Gini.

5.2.3 SVM Polinòmic

Un valor de C baix (0.1) indica que el model prefereix un marge més ample, encara que això impliqui cometre alguns errors en el conjunt d'entrenament. El fet que el grau 2 sigui millor que el grau 3 suggereix que la relació entre les variables (com l'hora, l'edat o el tipus de via) i la mortalitat té una curvatura simple. Introduir un grau 3 probablement feia el model massa complex per a la quantitat de dades o el nivell de soroll que teniu, provocant un lleuger overfitting.

Aquest model sembla estar esbiaixat cap a predir la classe minoritària, generant molt "soroll". Destaca pel seu Recall_min (0.75), el segon més alt. Detecta 3 de cada 4 casos minoritaris. No obstant això, la seva Accuracy cau a 0.43 i la f1_classmaj a 0.53.

Això indica que el model està classificant erròniament moltes mostres de la classe majoritària com a minoritària (massa falsos positius).

El MCC (0.09) és molt baix, indicant una correlació molt pobre entre la predicció i la realitat. La Balanced Accuracy (0.56) mostra que, tot i detectar la classe petita, el rendiment global és mediocre.

5.2.4 SVM RBF (Radial Basis Function)

Amb el nucli RBF, el model ha optat per un $C = 1$, una mica més alt. El valor de $\gamma = 0.1$ suggereix que la influència d'un sol punt d'entrenament arriba a una distància moderada. Presenta una Accuracy de 0.53 i un Recall_min de 0.58. No és ni prou precís amb la majoria ni prou sensible amb la minoria.

El seu f1_classmin (0.26) és inferior als arbres de decisió, indicant una pitjor relació entre precisió i recall per a la classe d'interès.

Amb un MCC de 0.07 i un ROC AUC de 0.55, aquest model aporta poc valor comparat amb els Arbres de Decisió. És pràcticament equivalent a l'atzar

5.2.5 SVM Lineal

Un valor de C baix (0.1) indica que el model prefereix un marge més ample, encara que això impliqui cometre alguns errors en el conjunt d'entrenament. Té el Recall_min més alt (0.82) de tots. Detecta el 82% dels casos buscats. Però la seva Accuracy és la pitjor (0.34) i el f1_classmaj (0.40) és desastrós.

El model està predient la classe minoritària gairebé sempre. Per això encerta la minoria (Recall alt) però falla massivament en la majoria (Accuracy baixa).

L'MCC (0.06) és el segon pitjor. La Balanced Accuracy (0.54) confirma que l'alt recall és enganyós i fruit d'un biaix massiu, no d'un aprenentatge real de patrons.

5.2.6 k-Nearest Neighbors (k-NN)

El k òptim és el valor més alt de la graella que provem. Això ens indica que el model necessita molts veïns per "suavitzar" el soroll de les dades i no deixar-se portar per casos aïllats, intentant trobar una tendència més estable dins el gran desequilibri de classes. Ha resultat que la distància de Manhattan dona millors resultats que l'Euclidiana, això és probablement degut a que la distància de Manhattan sol ser més robusta i efectiva en espais de dades categòriques.

Aquest és un cas de la paradoxa de l'accuracy. A primera vista, té la millor Accuracy (0.85) i el millor f1_classmaj (0.92).

Però si mirem el Recall_min (0.003) i el f1_classmin (0.007), veiem que el model no funciona bé. Ha decidit classificar-ho tot com a classe majoritària.

La Balanced Accuracy és 0.50 (el mateix que l'atzar) i l'MCC és 0.01 (correlació nul·la).

Aquest model ha fallat completament en la tasca d'aprendre a distingir les classes. Simplement ha après que "estadísticament surt a compte dir sempre que és la classe majoritària".

En la matriu de confusió pràcticament no detectem cap mort (només 2 de 536). El k-NN no té un paràmetre fàcil de "pes de classe". El k-NN és el pitjor model per a aquest problema precisament per això.

5.3 Interpretació dels resultats i implicacions dels resultats en el context del conjunt de dades utilitzat.

El millor model és del Decision Tree obtingut amb l'índex de Gini i amb una profunditat de 4. Per dir que és el millor observem que és el que té una AUC més alta i en aquest cas és la mètrica que hem considerat més important. Aquest model ens permet identificar una jerarquia clara de factors que determinen la gravetat d'un accident. El model no actua de manera genèrica, sinó que bifurca la seva anàlisi segons el tipus d'entorn, prioritzant diferents variables en funció de si l'accident és urbà o interurbà.

La Via Urbana s'estableix com el node arrel (el més important) amb un pes del 49,1 %. Aquesta és la gran línia divisòria: el model entén que la dinàmica d'un accident canvia completament segons si es produeix dins o fora de la ciutat.

Quan l'accident **no ocorre en zona urbana**, el risc es defineix per la física de l'impacte:

La velocitat de la via (major o menor que 55) és el segon factor de pes (15,6 %). En carretera, la velocitat és el principal predictor de mortalitat; a més velocitat permesa, menys marge de supervivència en cas de col·lisió.

Hi ha una clara zona geografia crítica (Vallès Occidental i Oriental). El tercer nivell de l'arbre ens indica que, en carreteres interurbanes, el Vallès Occidental actua com una zona de risc destacada. Això pot atribuir-se a l'alta densitat de vies ràpides de gran capacitat (com l'AP-7 o la C-58) que creuen la comarca.

Quan l'accident **ocorre dins de l'entorn urbà**, la gravetat no depèn tant de la velocitat pura, sinó de la complexitat del sinistre:

El factor clau aquí és si hi ha més d'una unitat afectada. Això sol diferenciar els atropellaments o col·lisions entre vehicles (més greus) de les incidències simples.

Després el model identifica el Barcelonès i la velocitat urbana com a variables finals. En una zona tan densa com l'àrea metropolitana de Barcelona, la combinació de moltes unitats implicades i la velocitat de la via (vies ràpides urbanes com les Rondes o la Gran Via) incrementa el risc de víctimes mortals.

Al quart nivell de l'arbre continua aprofundint en la territorialitat, hora i els límits de velocitat, confirmant que són els factors amb més poder predictiu del conjunt de dades. Tot i que les divisions posteriors guanyen en precisió estadística, la interpretabilitat es manté coherent amb els nivells superiors.

6 Conclusions i Treball Futur

L'objectiu principal d'aquest estudi ha estat analitzar la capacitat de diferents models de classificació per predir la gravetat dels accidents de trànsit a Catalunya, centrant-nos específicament en la detecció d'accidents mortals. Després de l'anàlisi exhaustiva dels resultats, es poden extreure les següents conclusions:

L'estudi ha posat de manifest la gran dificultat que suposa treballar amb un conjunt de dades fortament desequilibrat (on la classe minoritària, els morts, és molt petita en comparació amb els ferits lleus o il·lesos). Algorismes com el k-NN i l'SVM s'han mostrat especialment sensibles a aquest fet, tendint a ignorar la classe minoritària. En contrast, l'Arbre de Decisió (Decision Tree) ha demostrat ser el model més robust, aconseguint un equilibri més eficaç i uns resultats superiors en termes de ROC-AUC (0,6315) i Recall.

El valor obtingut del coeficient de correlació de Matthews (MCC = 0,1860) és un indicador crític de la naturalesa del problema. Aquest valor baix confirma que les classes no són linealment separables. Existeix una superposició significativa en les característiques (features) dels accidents mortals i els no mortals; és a dir, situacions amb paràmetres d'entrada molt similars (com ara la via o la velocitat) poden acabar amb resultats de gravetat diferents. Això implica que no existeix una frontera de decisió senzilla i que el risc depèn de variables altament volàtils o no recollides totalment en el dataset.

Malgrat la superposició, l'anàlisi de la importància de les variables ha revelat que factors com la tipologia de la via (urbana/interurbana) i la velocitat de la via són els predictors amb més pes. El model ha estat capaç d'identificar que la gravetat es dispara en contextos de velocitats altes i en zones interurbanes, validant així la lògica física dels accidents de trànsit.

Aquest estudi estableix les bases per a futures investigacions, on es podrien aplicar diverses estratègies per millorar la capacitat predictiva i la precisió del sistema:

Atès l'èxit de l'arbre de decisió individual, el següent pas natural seria l'ús de Random Forest o Gradient Boosting (XGBoost). Aquests models podrien reduir el soroll i millorar l'estabilitat de la predicció combinant múltiples arbres, cosa que podria ajudar a definir millor les fronteres de decisió que ara resulten difuses.

En aquest treball, la selecció de variables s'ha basat en la intuïció i l'experiència prèvia. Un treball futur hauria d'incloure un estudi estadístic rigorós de correlació i informació mútua (Mutual Information) per identificar quines variables aporten realment valor predictiu. Això permetria descartar variables redundants que afegeixen soroll i potser recuperar-ne d'altres que s'havien eliminat inicialment però que podrien ser claus.

Per mitigar l'efecte del desequilibri, es proposa explorar mètodes més avançats de sobremostreig de la classe minoritària, com SMOTE (Synthetic Minority Over-sampling Technique), o l'ús de funcions de pèrdua cost-sensitives més agressives que penalitzin amb més força l'error en la predicció de morts.

Una altra limitació rellevant detectada durant l'estudi és la possible falta de representativitat temporal del conjunt de dades. Com s'ha observat a l'anàlisi exploratòria, el pic d'accidents registrat a la matinada (00:00 - 03:00) no es correspon amb els fluxos de trànsit reals de Catalunya.

Això ens fa sospitar que les dades podrien patir un biaix de selecció, on els accidents nocturns (sovint associats a factors de risc com la somnolència o l'alcohol) estan sobrerepresentats respecte als accidents diürns. Aquesta distorsió podria haver afectat la capacitat dels models, com l'Arbre de Decisió, per generalitzar correctament en escenaris diürns d'alta densitat de trànsit, ja que el model ha estat entrenat amb una proporció d'accidents nocturns irrealment

alta.

7 Annexos

Referències

- [1] Kaggle. Road Traffic Injuries/Deaths Catalonia 2010-20. Disponible a: <https://www.kaggle.com/datasets/jordigarciacastillon/road-traffic-injuries-deaths-catalonia-201020?resource=download>
- [2] Generalitat de Catalunya. Accidents de trànsit amb morts o ferits greus. Disponible a: <https://analisi.transparenciacatalunya.cat/Transport/Accidents-de-tr-nsit-amb-morts-o-ferits-greus-a-Ca/rmgc-ncpb>

0.1 Estudi Accidents de Trànsit a Catalunya

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm
from sklearn.datasets import make_blobs, load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score, confusion_matrix,
    ↪ classification_report
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score, recall_score,
    f1_score, matthews_corrcoef, balanced_accuracy_score, roc_auc_score,
    RocCurveDisplay
)

def plot_decision_boundary(clf, X, y, title=None, ax=None, plot_support=True):
    if ax is None:
        fig, ax = plt.subplots(figsize=(6,5))
        x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
        y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
        xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300), np.linspace(y_min,
    ↪ y_max, 300))
        Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
        Z = Z.reshape(xx.shape)
        ax.contour(xx, yy, Z, levels=[-1, 0, 1], linestyles=['--', '-', '--'])
        ax.scatter(X[:, 0], X[:, 1], c=y, cmap='coolwarm', edgecolors='k')
        if plot_support and hasattr(clf, 'support_vectors_'):
            ax.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
    ↪ s=100, marker='x')
        if title:
            ax.set_title(title)
        return ax
df = pd.read_csv("Accidents_de_tr_nsit_amb_morts_o_ferits_greus_a_Catalunya.csv")
    ↪
pd.set_option('display.max_columns', None)
df.head()
```

```
[2]: # Creem una nova columna anomenada 'Morts_Binaria'
df['Morts_Binaria'] = (df['F_MORTS'] > 0).astype(int)
#1 si hi ha morts. 0 si no n'hi ha
df.head()
```

```
[3]: #llista de les variables que estudiarem (no les estudiarem directament algunes
    ↪ les modificarem)
columnes_bones = ['Any', 'dat', 'D_TIPUS_VIA', 'nomCom', 'F_UNITATS_IMPLICADES',
    ↪ 'C_VELOCITAT_VIA', 'D_SENTITS_VIA', 'grupDiaLab', 'hor', 'Morts_Binaria' ]
```

```
# sobreescrivim el df només amb aquestes variables
df = df[columnes_bones]
#mostrem les primeres per veure les variables i com són
df.head()
```

```
[4]: # Canviem la data pel mes
df['dat'] = pd.to_datetime(df['dat'], dayfirst=True)
# Creem la nova columna 'Mes'
df['Mes'] = df['dat'].dt.month
# Borrem la columna que ens deia la data
columnes_fora = ['dat']
# la borrem
df = df.drop(columns=columnes_fora)
df.head()
```

```
[5]: columna_hhmm_str = df['hor'].astype(str).str.zfill(4)

hores = pd.to_numeric(columna_hhmm_str.str[:2])
minuts = pd.to_numeric(columna_hhmm_str.str[2:])

df['minuts_dia'] = (hores * 60) + minuts

df = df.drop(columns=['hor'])

df.head()
```

```
[6]: # ara ho convertim a sinus i cosinus per poder tenir distància petita entre el
↪minut 1439 (23:59) i 0 (00:00)
T_max = 1440
angle = 2 * np.pi * df['minuts_dia'] / T_max
df['hora_sin'] = np.sin(angle)
df['hora_cos'] = np.cos(angle)
df = df.drop(columns=['minuts_dia'])
df.head()
```

```
[7]: # ara convertim grupDiaLab a variable binaria 1=feiner, 0=CapDeSetmana
diccionari = {'Feiners': 1, 'CapDeSetmana': 0}
# apliquem el canvi
df['grupDiaLab'] = df['grupDiaLab'].map(diccionari)
df.head()
```

```
[8]: #convertim els sentits també a variable binaria
mapeig_sentits = {
    'Un sol sentit': 1,
    'Doble sentit': 2
}
df['sentit_via'] = df['D_SENTITS_VIA'].map(mapeig_sentits)
```

```
df = df.dropna(subset=['sentit_via'])
df = df.drop(columns=['D_SENTITS_VIA'])
print(df.shape)
print(df['C_VELOCITAT_VIA'].isna().sum()) # el nombre de files amb velocitat nula
df.head()
```

```
[9]: # notem que tenim observacions amb la velocitat de la via = 999, això deu ser un
      ↪ error, o que no se'n
      # coneix la velocitat
```

```
pd.set_option('display.max_columns', None)

df_filtrat = df[df['C_VELOCITAT_VIA'] > 120]
print(df_filtrat.shape)
df_filtrat
```

```
[10]: # eliminem aquestes observacions

df = df[df['C_VELOCITAT_VIA'] <= 120]
print(df.shape)
df.head()

# passem de (13851, 11) a (12321, 11), hem tret les 1520 files amb velocitat
      ↪ nula i les 10 amb velocitat 999
```

```
[11]: # Seleccionem només les columnes que ens interessin
resultat_describe_agrupat = df.groupby('Morts_Binaria')[['C_VELOCITAT_VIA',
      ↪ 'F_UNITATS_IMPLICADES']].describe()
resultat_describe_agrupat
```

```
[11]:
```

	C_VELOCITAT_VIA							
	count	mean	std	min	25%	50%	75%	\
Morts_Binaria								
0	10535.0	84.553299	24.997425	0.0	70.0	100.0	100.0	
1	1786.0	87.922732	21.299430	20.0	80.0	100.0	100.0	

	F_UNITATS_IMPLICADES							
	max	count	mean	std	min	25%	50%	\
Morts_Binaria								
0	120.0	10535.0	1.910014	0.737834	1.0	2.0	2.0	
1	120.0	1786.0	1.949608	1.051467	1.0	1.0	2.0	

	75%	max
Morts_Binaria		
0	2.0	13.0

1 2.0 21.0

```
[12]: S_mean = -0.037852 # mean(hora_sin)
      C_mean = 0.138329 # mean(hora_cos)

      # 1. Càlcul de l'Angle (en radians)
      angle_radians = np.arctan2(S_mean, C_mean)

      # 2. Convertir l'angle a minuts del dia (0 a 1440)
      # Afegeix 2*pi per assegurar que és positiu abans d'escalar
      minuts_dia = (angle_radians % (2 * np.pi)) / (2 * np.pi) * 1440

      # 3. Convertir a format Hores:Minuts
      hores = int(minuts_dia // 60)
      minuts = int(minuts_dia % 60)

      print(f"L'hora mitjana circular del grup 0 és: {hores:02d}:{minuts:02d}h")
```

L'hora mitjana circular del grup 0 és: 22:58h

```
[13]: S_mean = -0.034944 # mean(hora_sin)
      C_mean = 0.094747 # mean(hora_cos)

      # 1. Càlcul de l'Angle (en radians)
      angle_radians = np.arctan2(S_mean, C_mean)

      # 2. Convertir l'angle a minuts del dia (0 a 1440)
      # Afegeix 2*pi per assegurar que és positiu abans d'escalar
      minuts_dia = (angle_radians % (2 * np.pi)) / (2 * np.pi) * 1440

      # 3. Convertir a format Hores:Minuts
      hores = int(minuts_dia // 60)
      minuts = int(minuts_dia % 60)

      print(f"L'hora mitjana circular del grup 1 és: {hores:02d}:{minuts:02d}h")
```

```
[14]: print(df.head())
```

	Any	D_TIPUS_VIA	nomCom	\
0	2010	Via urbana(inclou carrer i carrer residencial)	Valles Oriental	
1	2010	Carretera convencional	Segria	
2	2010	Carretera convencional	Girones	
3	2010	Via urbana(inclou carrer i carrer residencial)	Barcelones	
4	2010	Via urbana(inclou carrer i carrer residencial)	Barcelones	

	F_UNITATS_IMPLICADES	C_VELOCITAT_VIA	grupDiaLab	Morts_Binaria	Mes	\
0	2	100.0	1	0	1	
1	1	40.0	0	0	10	

2	4	80.0	1	1	5
3	2	100.0	0	0	8
4	1	100.0	0	0	5

	hora_sin	hora_cos	sentit_via
0	-0.117537	0.993068	1.0
1	0.004363	0.999990	2.0
2	-0.785317	-0.619094	2.0
3	0.584250	0.811574	1.0
4	-0.997859	-0.065403	1.0

1 Aplicació d'algorismes d'aprenentatge automàtic

```
[15]: def metrics(y_true, y_pred):
    return dict(
        accuracy=accuracy_score(y_true, y_pred),
        recall_min=recall_score(y_true, y_pred), # recall classe 1 que és la
        ↳minoritaria
        f1_classmin=f1_score(y_true, y_pred),
        f1_classmaj=f1_score(y_true, y_pred, pos_label=0),
        balanced_accuracy=balanced_accuracy_score(y_true, y_pred),
        mcc=matthews_corrcoef(y_true, y_pred),
        roc_auc= roc_auc_score(y_true,y_pred)
    )
```

1.1 Support Vector Machines

1.1.1 kernel= lineal

```
[16]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import svm
from sklearn.datasets import make_blobs, load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score, confusion_matrix,
↳classification_report
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

SVC= svm.SVC

columnes_a_eliminar = ['D_TIPUS_VIA', 'nomCom']
X = df.drop(columns=['Morts_Binaria'] + columnes_a_eliminar)
y = df['Morts_Binaria']
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
↳random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

param_grid_linear = {'C': [0.1, 1, 10, 100], 'kernel': ['linear']}

grid_linear = GridSearchCV(
    SVC(class_weight='balanced', max_iter=20000), # balanced això ajuda el
↳model a no ignorar la classe minoritària
    param_grid_linear,
    scoring='roc_auc', # usem roc_auc (accuracy no adequada pel desequilibri en
↳les dades)
    verbose=0,
    cv=5 # cross validation
)
grid_linear.fit(X_train_scaled, y_train)

print("--- MILLOR MODEL LINEAL ---")
print(grid_linear.best_params_)
pred_linear = grid_linear.predict(X_test_scaled)
print(confusion_matrix(y_test, pred_linear))
metrics(y_test, pred_linear)

```

```

--- MILLOR MODEL LINEAL ---
{'C': 0.1, 'kernel': 'linear'}
[[ 822 2339]
 [  95 441]]

```

```

[16]: {'accuracy': 0.3416283473086286,
      'recall_min': 0.8227611940298507,
      'f1_classmin': 0.2659831121833534,
      'f1_classmaj': 0.4031387935262384,
      'balanced_accuracy': 0.5414027419057826,
      'mcc': 0.06750666852616789,
      'roc_auc': 0.5414027419057827}

```

1.1.2 kernel = polinòmic

```

[17]: param_grid_poly = {
      'C': [0.1, 1, 10, 100],
      'degree': [2, 3],
      'kernel': ['poly']
    }

```

```

grid_poly = GridSearchCV(
    SVC(class_weight='balanced'),
    param_grid_poly,
    scoring='roc_auc',
    verbose=0,
    cv=5
)

grid_poly.fit(X_train_scaled, y_train)

print("--- MILLOR MODEL POLINÒMIC ---")
print(f"Millors paràmetres: {grid_poly.best_params_}")

pred_poly = grid_poly.predict(X_test_scaled)
print(confusion_matrix(y_test, pred_poly))
metrics(y_test, pred_poly)

```

```

--- MILLOR MODEL POLINÒMIC ---
Millors paràmetres: {'C': 0.1, 'degree': 2, 'kernel': 'poly'}
[[1184 1977]
 [ 134  402]]

```

```

[17]: {'accuracy': 0.42899648363538,
      'recall_min': 0.75,
      'f1_classmin': 0.2758147512864494,
      'f1_classmaj': 0.5286894396070552,
      'balanced_accuracy': 0.5622825055362227,
      'mcc': 0.09156632571275168,
      'roc_auc': 0.5622825055362227}

```

1.1.3 kernel = rbf

```

[18]: param_grid_rbf = {
      'C': [0.1, 1, 10, 100],
      'gamma': [0.1, 0.01, 0.001],
      'kernel': ['rbf']
    }

grid_rbf = GridSearchCV(
    SVC(class_weight='balanced'),
    param_grid_rbf,
    scoring='roc_auc',
    verbose=0,
    cv=5
)

grid_rbf.fit(X_train_scaled, y_train)

```

```

print("--- MILLOR MODEL RBF ---")
print(f"Millors paràmetres: {grid_rbf.best_params_}")

pred_rbf = grid_rbf.predict(X_test_scaled)
print(confusion_matrix(y_test, pred_rbf))
metrics(y_test, pred_rbf)

```

--- MILLOR MODEL RBF ---

```

Millors paràmetres: {'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}
[[1668 1493]
 [ 223  313]]

```

```

[18]: {'accuracy': 0.5358398701649987,
      'recall_min': 0.5839552238805971,
      'f1_classmin': 0.26729291204099065,
      'f1_classmaj': 0.6603325415676959,
      'balanced_accuracy': 0.5558181687261259,
      'mcc': 0.07863132340770546,
      'roc_auc': 0.5558181687261259}

```

1.2 k-Nearest Neighbours

```

[19]: from sklearn.neighbors import KNeighborsClassifier
      from sklearn.metrics import balanced_accuracy_score, matthews_corrcoef
      from sklearn.preprocessing import MinMaxScaler

df_knn = pd.get_dummies(df, columns=['D_TIPUS_VIA', 'nomCom'])
X = df_knn.drop(columns='Morts_Binaria')
y = df_knn['Morts_Binaria']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Usem este escalador pq tenim categoriques
scaler_minmax = MinMaxScaler()
X_train_knn = scaler_minmax.fit_transform(X_train)
X_test_knn = scaler_minmax.transform(X_test)

```

```

[20]: param_grid = {'n_neighbors': [1, 3, 5, 7, 9, 11, 15, 21, 25],
                    'p': [1, 2], # p=1 és Manhattan, p=2 és Euclidiana
                    }

grid_search = GridSearchCV(
    KNeighborsClassifier(),
    param_grid,
    scoring='roc_auc',
    cv=5
)

```



```

)

grid_search.fit(X_train_knn, y_train)
best_knn = grid_search.best_estimator_

print(f"Millor valor de k trobat: {grid_search.best_params_['n_neighbors']}")
print(f"Millor distància: {grid_search.best_params_['p']}")
print(f"Millor ROC-AUC en validació: {grid_search.best_score_:.4f}")

pred_knn = best_knn.predict(X_test_knn)

print(confusion_matrix(y_test, pred_knn))
metrics(y_test, pred_knn)

```

```

Millor valor de k trobat: 25
Millor distància: 1
Millor ROC-AUC en validació: 0.6402
[[3154    7]
 [ 534    2]]

```

```

[20]: {'accuracy': 0.8536651338923451,
      'recall_min': 0.0037313432835820895,
      'f1_classmin': 0.007339449541284403,
      'f1_classmaj': 0.921010366476858,
      'balanced_accuracy': 0.5007584270989248,
      'mcc': 0.010837316816533937,
      'roc_auc': 0.5007584270989248}

```

1.3 Decision Tree

```

[21]: from sklearn.tree import DecisionTreeClassifier, plot_tree
      from sklearn.metrics import ConfusionMatrixDisplay

      df_dt = pd.get_dummies(df, columns=['D_TIPUS_VIA', 'nomCom'], drop_first=True)

      X = df_dt.drop(columns='Morts_Binaria')
      y = df_dt['Morts_Binaria']

      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
      ↪random_state=42, stratify=y)

```

```

[22]: from sklearn.model_selection import cross_val_score

      depths = range(1, 15)
      roc_gini, roc_entropy = [], []

      for d in depths:

```

```

    model_g = DecisionTreeClassifier(criterion='gini', max_depth=d,
↪class_weight='balanced', random_state=42)
    roc_gini.append(cross_val_score(model_g, X_train, y_train, cv=5,
↪scoring='roc_auc').mean())

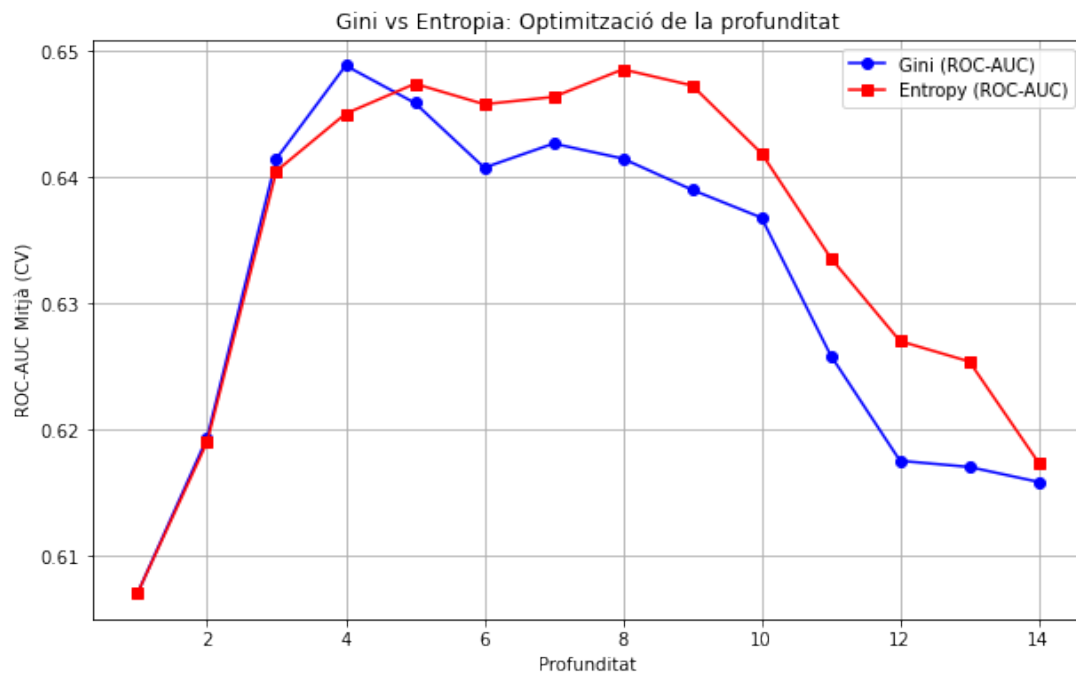
    model_e = DecisionTreeClassifier(criterion='entropy', max_depth=d,
↪class_weight='balanced', random_state=42)
    roc_entropy.append(cross_val_score(model_e, X_train, y_train, cv=5,
↪scoring='roc_auc').mean())

plt.figure(figsize=(10, 6))
plt.plot(depths, roc_gini, marker='o', label='Gini (ROC-AUC)', color='blue')
plt.plot(depths, roc_entropy, marker='s', label='Entropy (ROC-AUC)', color='red')
plt.xlabel('Profunditat')
plt.ylabel('ROC-AUC Mitjà (CV)')
plt.title('Gini vs Entropia: Optimització de la profunditat')
plt.legend()
plt.grid(True)
plt.show()

best_g_idx, best_e_idx = np.argmax(roc_gini), np.argmax(roc_entropy)

print(f"Millor ROC-AUC Gini: {roc_gini[best_g_idx]:.4f} (Profunditat:
↪{depths[best_g_idx]})")
print(f"Millor ROC-AUC Entropy: {roc_entropy[best_e_idx]:.4f} (Profunditat:
↪{depths[best_e_idx]})")

```



Millor ROC-AUC Gini: 0.6488 (Profunditat: 4)

Millor ROC-AUC Entropy: 0.6485 (Profunditat: 8)

Usam Gini

```
[23]: clf_gini = DecisionTreeClassifier(
        criterion='gini',
        max_depth=4,
        class_weight='balanced',
        random_state=42
    )
    clf_gini.fit(X_train, y_train)

    importances = pd.DataFrame({
        'feature': X.columns,
        'importance': clf_gini.feature_importances_
    }).sort_values('importance', ascending=False)

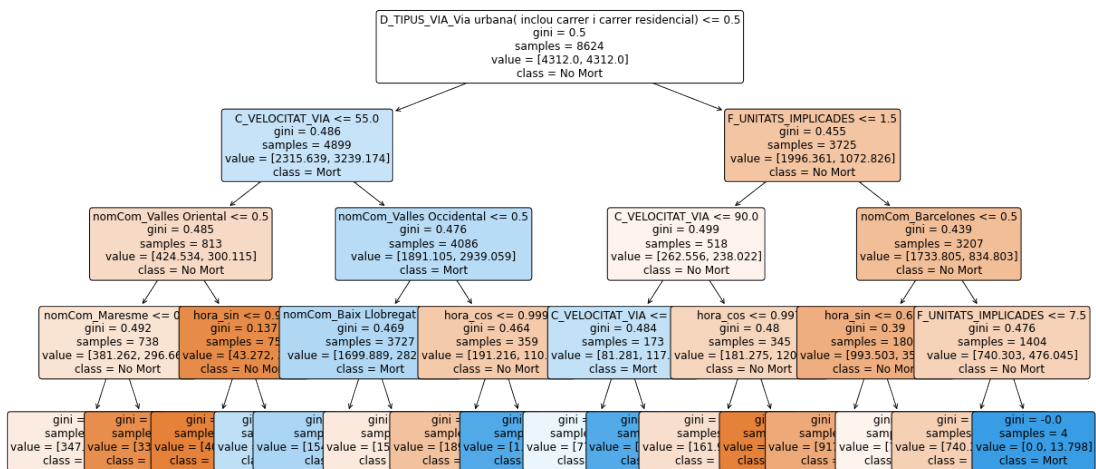
    print(" 3 variables més importants")
    print(importances.head(3))

    plt.figure(figsize=(20, 10))
    plot_tree(
        clf_gini,
        feature_names=X.columns,
```

```
class_names=['No Mort', 'Mort'],
filled=True,
rounded=True,
fontsize=12
)
plt.show()
```

3 variables més importants

	feature	importance
12	D_TIPUS_VIA_Via urbana(inclou carrer i carrer...	0.490957
2	C_VELOCITAT_VIA	0.144847
52	nomCom_Valles Occidental	0.086092



1.3.1 Usem entropia

```
[24]: clf_entropia = DecisionTreeClassifier(
        criterion='entropy',
        max_depth=5,
        class_weight='balanced',
        random_state=42
    )

    clf_entropia.fit(X_train, y_train)

    importances = pd.DataFrame({
        'feature': X.columns,
        'importance': clf_entropia.feature_importances_
    }).sort_values('importance', ascending=False)

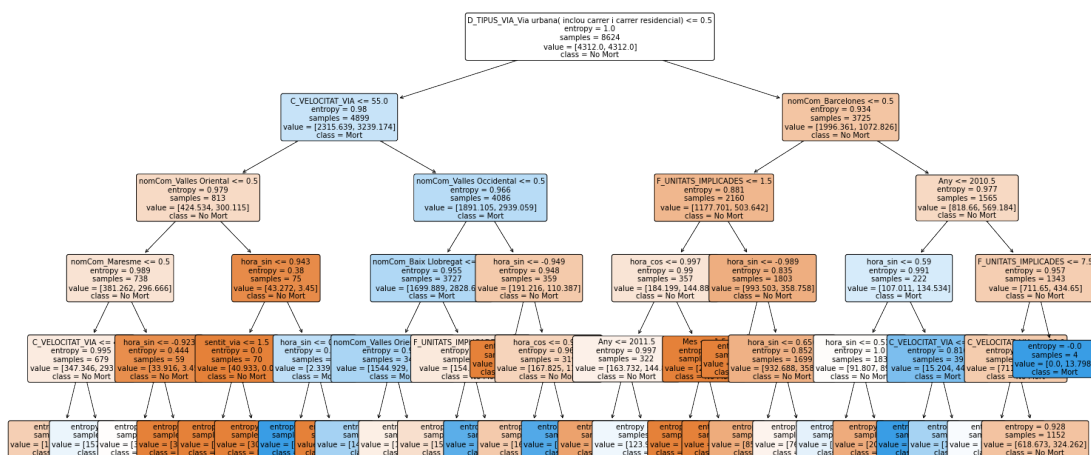
    print(" 4 variables m s importants")
```

```
print(importances.head(4))

plt.figure(figsize=(25, 12))
plot_tree(
    clf_entropia,
    feature_names=X.columns,
    class_names=['No Mort', 'Mort'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.show()
```

4 variables més importants

	feature	importance
12	D_TIPUS_VIA_Via urbana(inclou carrer i carrer...	0.387496
5	hora_sin	0.131526
2	C_VELOCITAT_VIA	0.124476
52	nomCom_Valles Occidental	0.068436



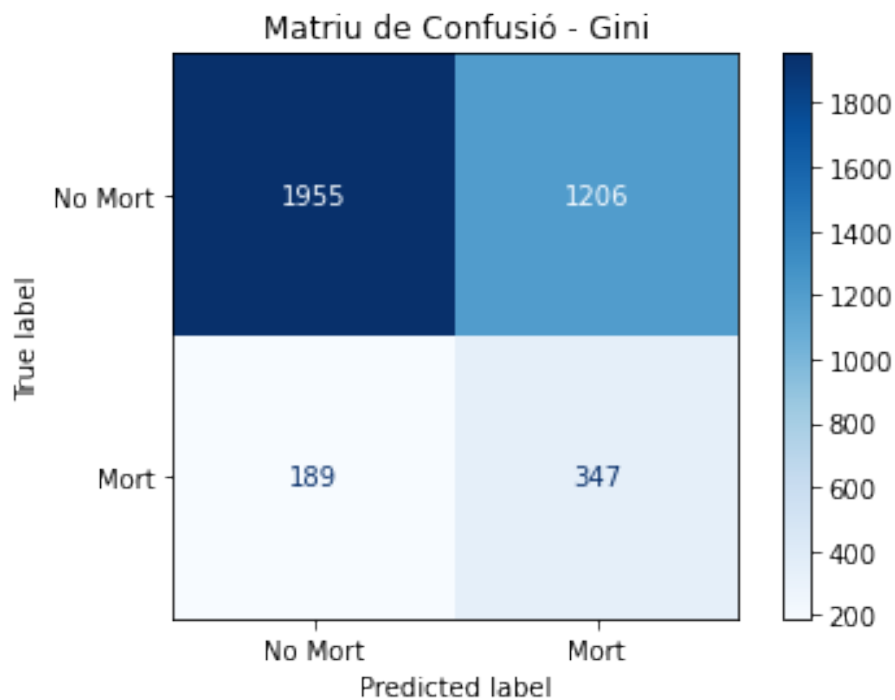
```
[25]: for name, model, color in [("Gini", clf_gini, 'Blues'), ("Entropy",
    ↪clf_entropia, 'Greens')]:
    y_pred = model.predict(X_test)

    print(f"\n RESULTATS MODEL {name.upper()} ")
    print(metrics(y_test,y_pred))
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['NoM
    ↪ort', 'Mort'])
    disp.plot(cmap=color)
```

```
plt.title(f"Matriu de Confusió - {name}")
plt.show()
```

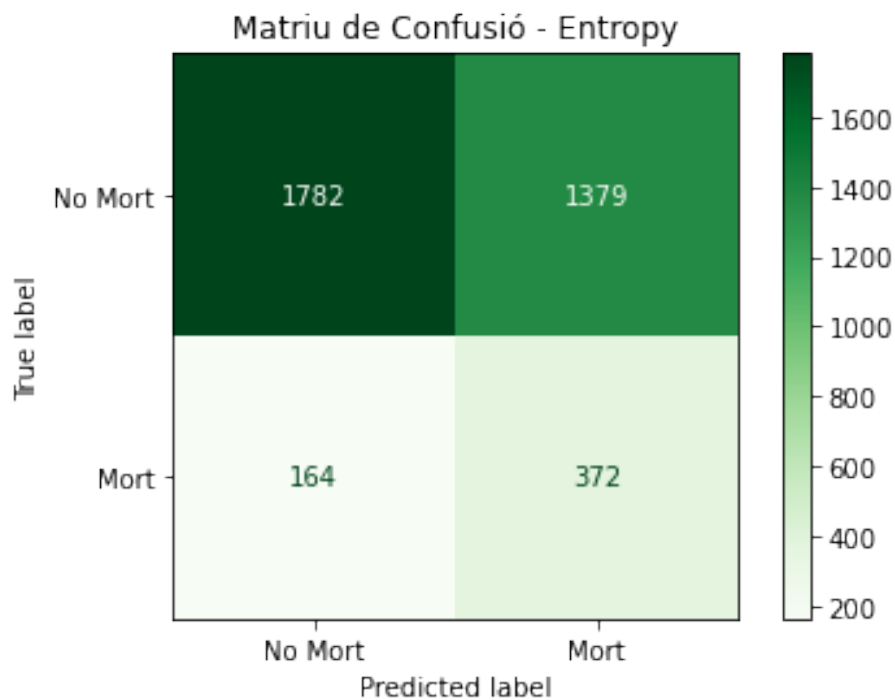
RESULTATS MODEL GINI

```
{'accuracy': 0.6226670273194482, 'recall_min': 0.6473880597014925,
'f1_classmin': 0.3322163714696027, 'f1_classmaj': 0.7370405278039585,
'balanced_accuracy': 0.632931612894087, 'mcc': 0.189650850319239, 'roc_auc':
0.632931612894087}
```



RESULTATS MODEL ENTROPY

```
{'accuracy': 0.5826345685691101, 'recall_min': 0.6940298507462687,
'f1_classmin': 0.32531700918233497, 'f1_classmaj': 0.6978656745643234,
'balanced_accuracy': 0.6288877504284965, 'mcc': 0.18176982584716858, 'roc_auc':
0.6288877504284967}
```



1.4 Resultats finals

```
[26]: from sklearn.metrics import roc_auc_score, balanced_accuracy_score, \
      ↪matthews_corrcoef

# creem una llista amb els models i les seves prediccions
models_eval = [
    ("SVM Lineal", pred_linear),
    ("SVM Polinòmic", pred_poly),
    ("SVM RBF", pred_rbf),
    ("k-Nearest Neighbors", pred_knn),
    ("Decision Tree (Gini)", clf_gini.predict(X_test)),
    ("Decision Tree (Entropy)", clf_entropia.predict(X_test))
]

# calculem mètriques per a cada model
results = []

for name, preds in models_eval:
    m = metrics(y_test, preds)
    results.append({
        "Model": name,
        **m
    })
```

```

    })

    # convertim a DataFrame i ordenem pel millor roc_auc
    df_results = pd.DataFrame(results).sort_values(by="roc_auc", ascending=False)

    print("--- COMPARATIVA FINAL DE MODELS ---")
    print(df_results.to_string(index=False))

```

```

--- COMPARATIVA FINAL DE MODELS ---
      Model accuracy recall_min f1_classmin f1_classmaj
balanced_accuracy      mcc roc_auc
Decision Tree (Gini)  0.622667   0.647388   0.332216   0.737041
0.632932 0.189651 0.632932
Decision Tree (Entropy) 0.582635   0.694030   0.325317   0.697866
0.628888 0.181770 0.628888
      SVM Polinòmic 0.428996   0.750000   0.275815   0.528689
0.562283 0.091566 0.562283
      SVM RBF 0.535840   0.583955   0.267293   0.660333
0.555818 0.078631 0.555818
      SVM Lineal 0.341628   0.822761   0.265983   0.403139
0.541403 0.067507 0.541403
      k-Nearest Neighbors 0.853665   0.003731   0.007339   0.921010
0.500758 0.010837 0.500758

```

GRÀFIC 1: Dia Laborable vs Cap de Setmana

```

[27]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df_plot = df.copy()

df_plot['Resultat'] = df_plot['Morts_Binaria'].replace({
    0: 'Sense Morts (Y=0)',
    1: 'Amb Morts (Y=1)'
})

df_plot['grupDiaLab_etiqueta'] = df_plot['grupDiaLab'].replace({
    0: 'Cap de Setmana',
    1: 'Feiners'
})

plt.figure(figsize=(8, 6))

sns.countplot(
    x='grupDiaLab_etiqueta',

```



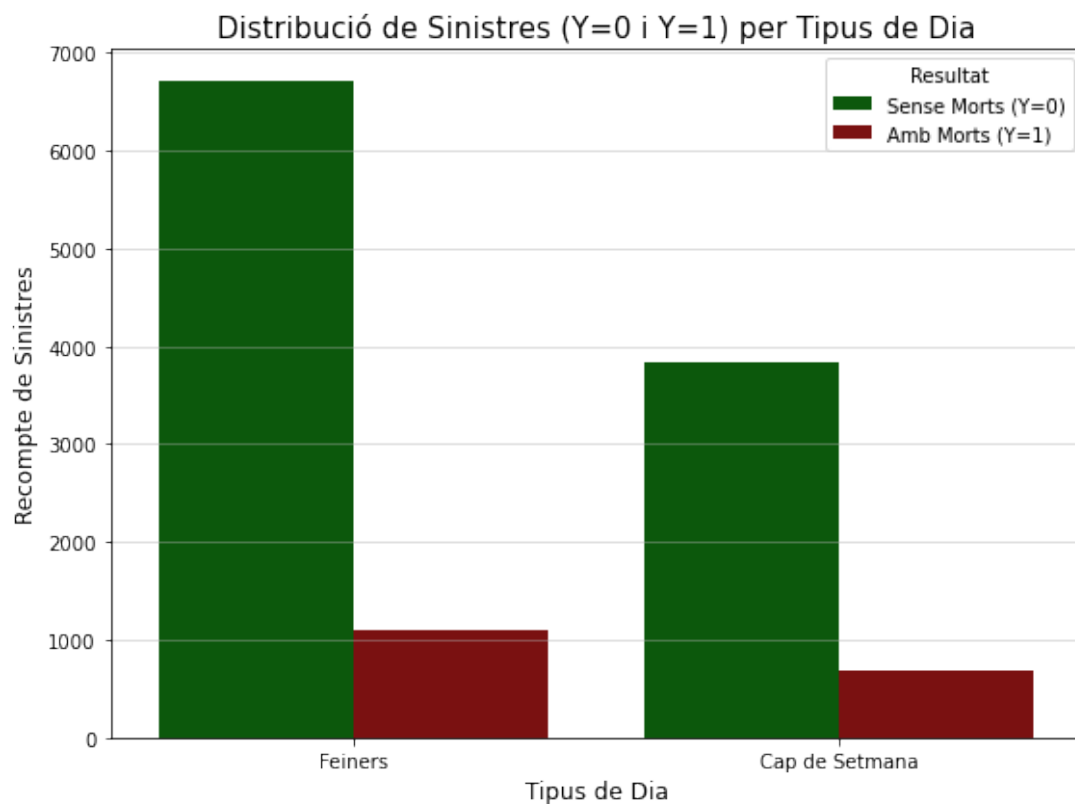
```

hue='Resultat',
data=df_plot,
palette={'Sense Morts (Y=0)': 'darkgreen', 'Amb Morts (Y=1)': 'darkred'}
)

plt.title('Distribució de Sinistres (Y=0 i Y=1) per Tipus de Dia', fontsize=15)
plt.xlabel('Tipus de Dia', fontsize=12)
plt.ylabel('Recompte de Sinistres', fontsize=12)
plt.legend(title='Resultat')

plt.grid(axis='y', alpha=0.5)
plt.tight_layout()
plt.show()

```



GRÀFIC 2: Sentit de la Via

```

[28]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df_plot = df.copy()

```

```

df_plot['Resultat'] = df_plot['Morts_Binaria'].replace({
    0: 'Sense Morts (Y=0)',
    1: 'Amb Morts (Y=1)'
})

df_plot['sentit_via_etiqueta'] = df_plot['sentit_via'].replace({
    1.0: 'Sentit Únic',
    2.0: 'Doble Sentit'
})

plt.figure(figsize=(8, 6))

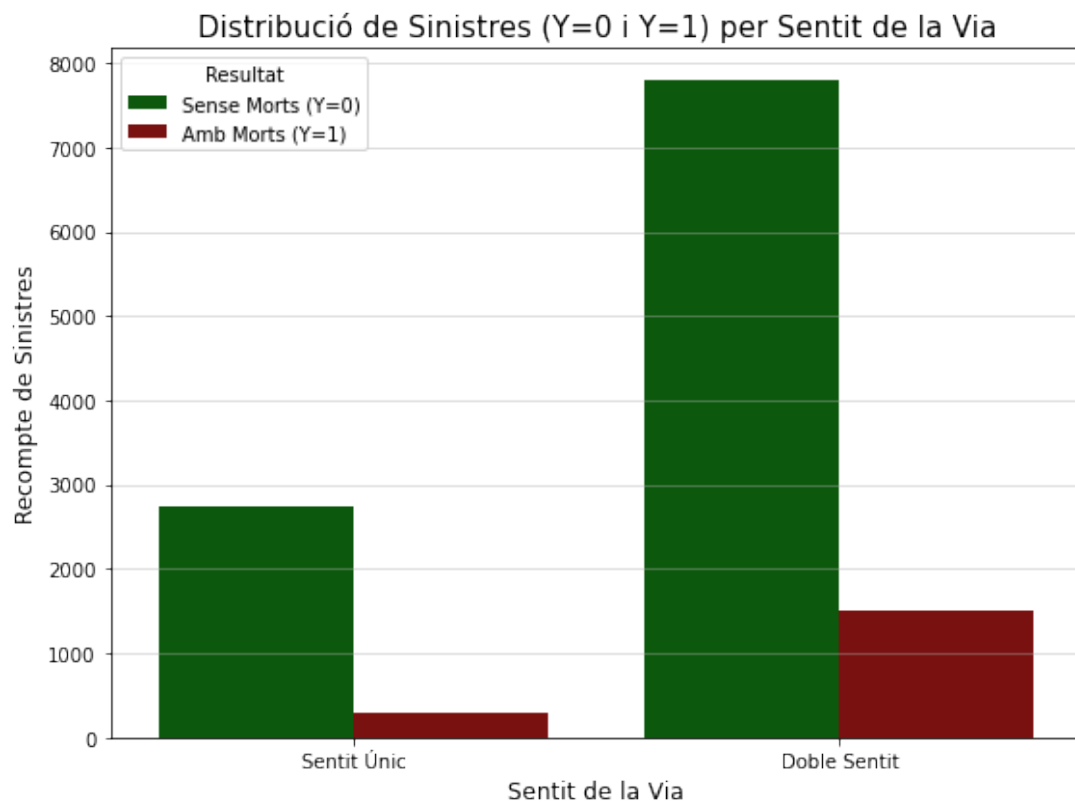
sns.countplot(
    x='sentit_via_etiqueta',
    hue='Resultat',
    data=df_plot,

    palette={'Sense Morts (Y=0)': 'darkgreen', 'Amb Morts (Y=1)': 'darkred'}
)

plt.title('Distribució de Sinistres (Y=0 i Y=1) per Sentit de la Via',
    ↪fontsize=15)
plt.xlabel('Sentit de la Via', fontsize=12)
plt.ylabel('Recompte de Sinistres', fontsize=12)
plt.legend(title='Resultat')

plt.grid(axis='y', alpha=0.5)
plt.tight_layout()
plt.show()

```



GRÀFIC 3: Tipus de Via

```
[29]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df_plot = df.copy()
df_plot['Resultat'] = df_plot['Morts_Binaria'].replace({
    0: 'Sense Morts (Y=0)',
    1: 'Amb Morts (Y=1)'
})

df_y1 = df_plot[df_plot['Morts_Binaria'] == 1]
tipus_via_order_y1 = df_y1['D_TIPUS_VIA'].value_counts().index.tolist()

plt.figure(figsize=(10, 7))

sns.countplot(
    y='D_TIPUS_VIA',
```

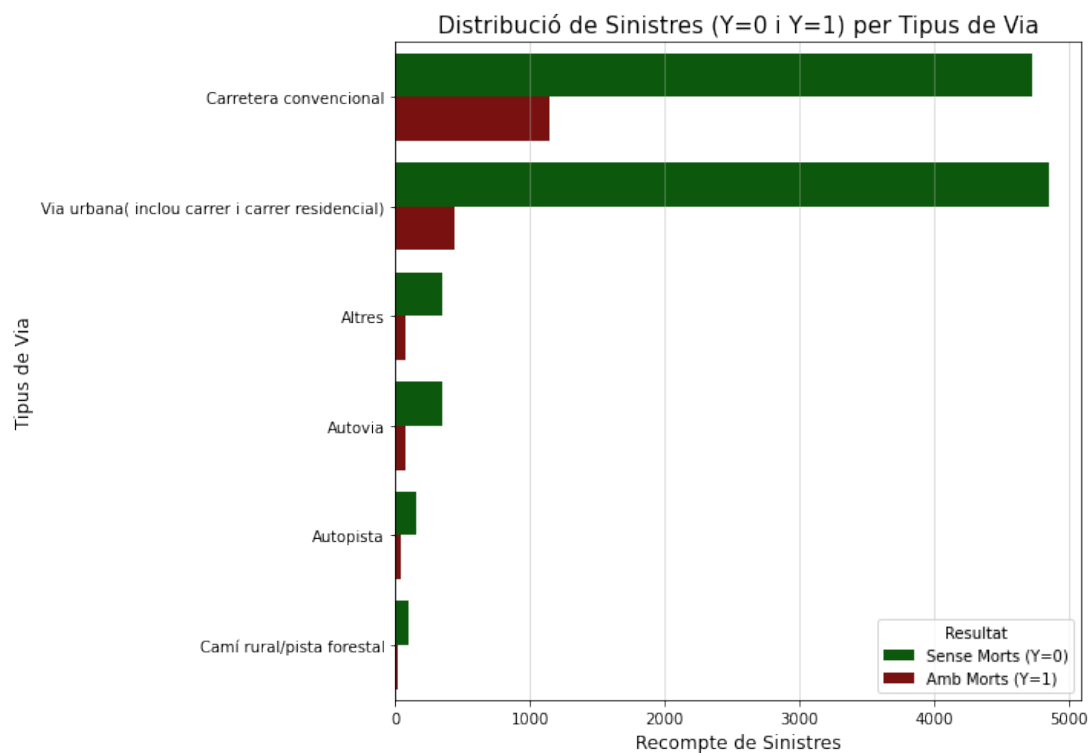
```

hue='Resultat',
data=df_plot,
order=tipus_via_order_y1,
palette={'Sense Morts (Y=0)': 'darkgreen', 'Amb Morts (Y=1)': 'darkred'}
)

plt.title('Distribució de Sinistres (Y=0 i Y=1) per Tipus de Via', fontsize=15)
plt.xlabel('Recompte de Sinistres', fontsize=12)
plt.ylabel('Tipus de Via', fontsize=12)
plt.legend(title='Resultat')

plt.grid(axis='x', alpha=0.5)
plt.tight_layout()
plt.show()

```



GRÀFIC 4: Top 10 Comarques amb més accidents

```

[30]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

```

```

df_plot = df.copy()

df_plot['Resultat'] = df_plot['Morts_Binaria'].replace({
    0: 'Sense Morts (Y=0)',
    1: 'Amb Morts (Y=1)'
})

df_només_morts = df_plot[df_plot['Morts_Binaria'] == 1]

top_10_comarques_mortals = df_només_morts['nomCom'].value_counts().index[:10].
    ↪tolist()

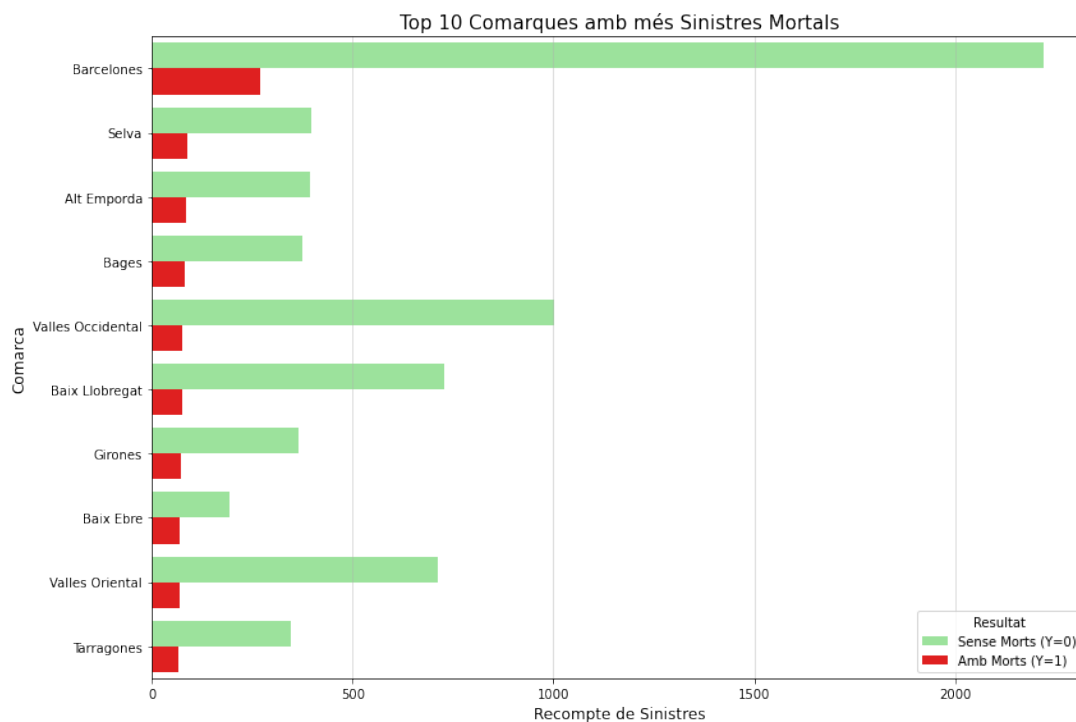
plt.figure(figsize=(12, 8))

sns.countplot(
    y='nomCom',
    hue='Resultat',
    data=df_plot,
    order=top_10_comarques_mortals,
    palette={'Sense Morts (Y=0)': 'lightgreen', 'Amb Morts (Y=1)': 'red'}
)

plt.title('Top 10 Comarques amb més Sinistres Mortals', fontsize=15)
plt.xlabel('Recompte de Sinistres', fontsize=12)
plt.ylabel('Comarca', fontsize=12)
plt.legend(title='Resultat')

plt.grid(axis='x', alpha=0.5)
plt.tight_layout()
plt.show()

```



GRÀFIC 5: Distribució del nombre d'accidents per any

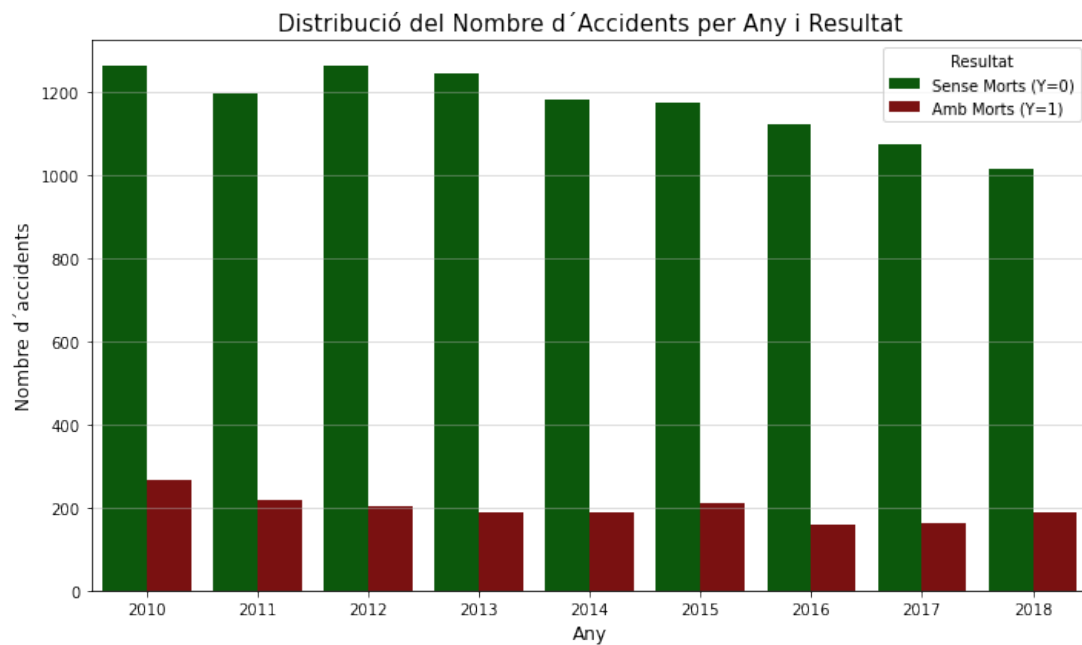
```
[31]: plt.figure(figsize=(10, 6))

df_plot = df.copy()
df_plot['Resultat'] = df_plot['Morts_Binaria'].replace({0: 'Sense Morts (Y=0)',
↪1: 'Amb Morts (Y=1)'})

sns.countplot(
    x='Any',
    hue='Resultat',
    data=df_plot,
    palette={'Sense Morts (Y=0)': 'darkgreen', 'Amb Morts (Y=1)': 'darkred'}
)

plt.title('Distribució del Nombre d'Accidents per Any i Resultat', fontsize=15)
plt.xlabel('Any', fontsize=12)
plt.ylabel('Nombre d'accidents', fontsize=12)
plt.legend(title='Resultat')

plt.grid(axis='y', alpha=0.5)
plt.tight_layout()
plt.show()
```



GRÀFIC 6: Distribució del nombre d'accidents per mes

```
[32]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df_plot = df.copy()

df_plot['Resultat'] = df_plot['Morts_Binaria'].replace({
    0: 'Sense Morts (Y=0)',
    1: 'Amb Morts (Y=1)'
})

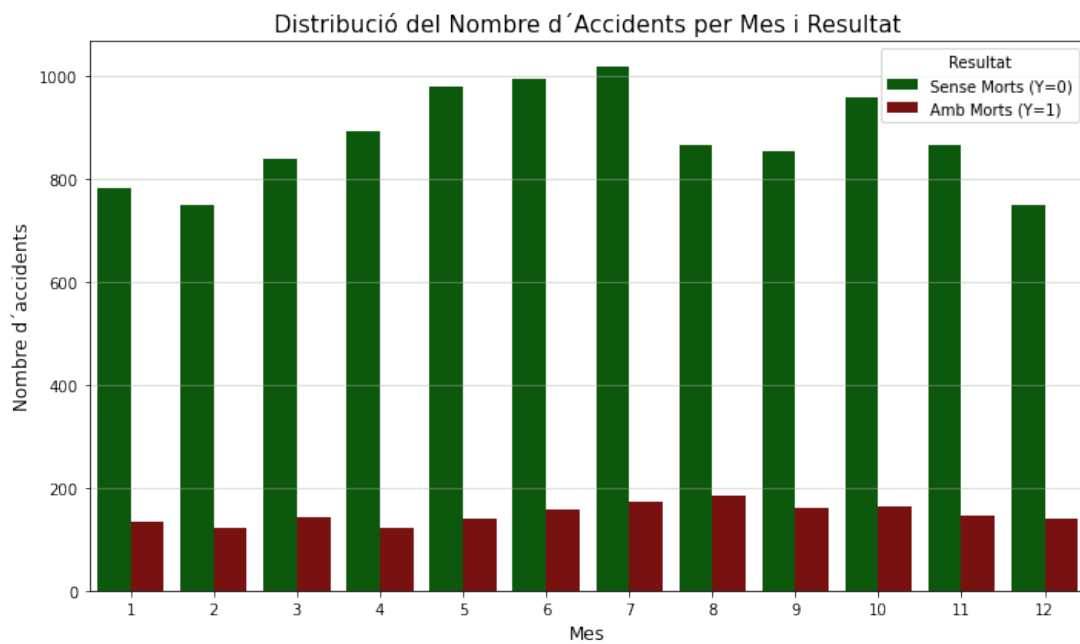
df_plot['Mes'] = df_plot['Mes'].astype('category')

plt.figure(figsize=(10, 6))

sns.countplot(
    x='Mes',
    hue='Resultat',
    data=df_plot,
    palette={'Sense Morts (Y=0)': 'darkgreen', 'Amb Morts (Y=1)': 'darkred'}
)
```

```
plt.title('Distribució del Nombre d'Accidents per Mes i Resultat', fontsize=15)
plt.xlabel('Mes', fontsize=12)
plt.ylabel('Nombre d'accidents', fontsize=12)
plt.legend(title='Resultat')

plt.grid(axis='y', alpha=0.5)
plt.tight_layout()
plt.show()
```



GRÀFIC 7: Nombre d'unitats implicades en els accidents

```
[33]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df_plot = df.copy()

df_plot['Resultat'] = df_plot['Morts_Binaria'].replace({
    0: 'Sense Morts (Y=0)',
    1: 'Amb Morts (Y=1)'
})
```



```

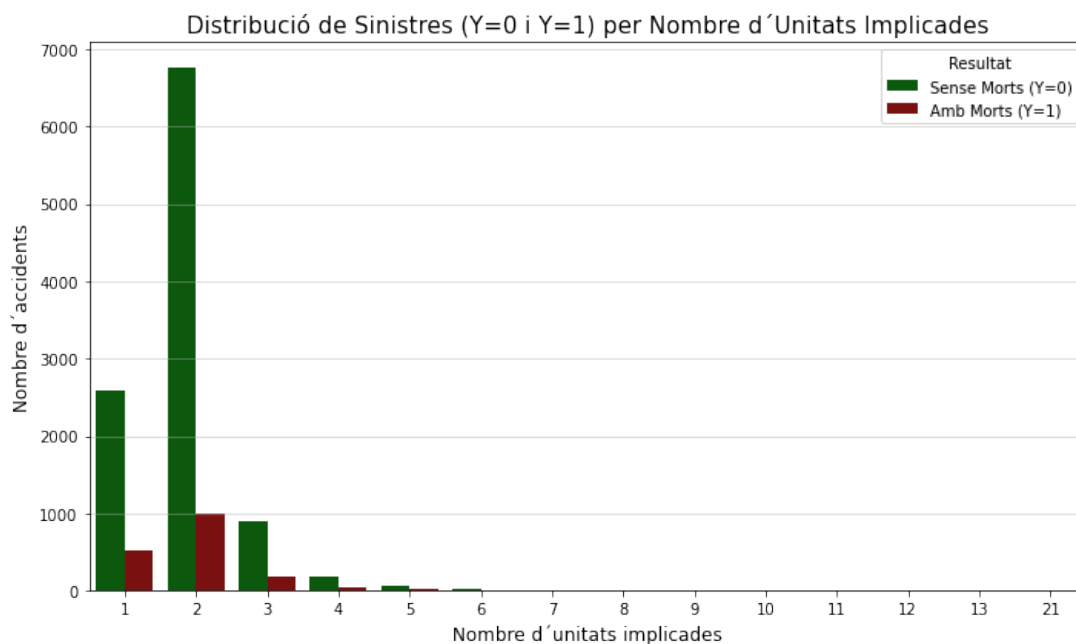
df_plot['F_UNITATS_IMPLICADES'] = df_plot['F_UNITATS_IMPLICADES'].
↳astype('category')

plt.figure(figsize=(10, 6))

sns.countplot(
    x='F_UNITATS_IMPLICADES',
    hue='Resultat',
    data=df_plot,
    palette={'Sense Morts (Y=0)': 'darkgreen', 'Amb Morts (Y=1)': 'darkred'}
)

plt.title('Distribució de Sinistres (Y=0 i Y=1) per Nombre d'Unitats_
↳Implicades', fontsize=15)
plt.xlabel('Nombre d'unitats implicades', fontsize=12)
plt.ylabel('Nombre d'accidents', fontsize=12)
plt.legend(title='Resultat')
plt.grid(axis='y', alpha=0.5)
plt.tight_layout()
plt.show()

```



GRÀFIC 8: Freqüència d'Accidents per tots els Límits de Velocitat de la Via

```

[34]: import pandas as pd
import seaborn as sns

```

```

import matplotlib.pyplot as plt

df_plot = df.copy()

df_plot['Resultat'] = df_plot['Morts_Binaria'].replace({
    0: 'Sense Morts (Y=0)',
    1: 'Amb Morts (Y=1)'
})

df_plot['C_VELOCITAT_VIA'] = df_plot['C_VELOCITAT_VIA'].astype('category')

plt.figure(figsize=(12, 6))

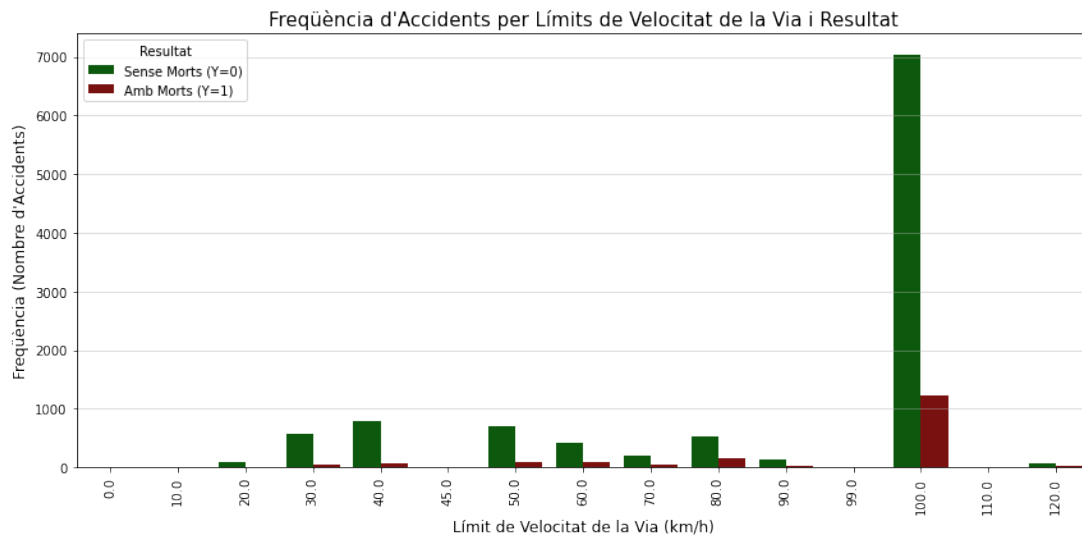
sns.countplot(
    x='C_VELOCITAT_VIA',
    hue='Resultat',
    data=df_plot,
    palette={'Sense Morts (Y=0)': 'darkgreen', 'Amb Morts (Y=1)': 'darkred'},
    order=sorted(df_plot['C_VELOCITAT_VIA'].unique())
)

plt.title('Freqüència d\'Accidents per Límits de Velocitat de la Via i  
↳Resultat', fontsize=15)
plt.xlabel('Límit de Velocitat de la Via (km/h)', fontsize=12)
plt.ylabel('Freqüència (Nombre d\'Accidents)', fontsize=12)

plt.xticks(rotation=90, ha='center')

plt.grid(axis='y', alpha=0.5)
plt.tight_layout()
plt.show()

```



GRÀFIC 9: Freqüència d'Accidents al Llarg del Dia

```
[35]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

df_y0 = df[df['Morts_Binaria'] == 0]
df_y1 = df[df['Morts_Binaria'] == 1]

def calculate_angle(df_subset):
    angle_rad = np.arctan2(df_subset['hora_sin'], df_subset['hora_cos'])
    # Assegurar que l'angle és positiu (de 0 a 2*pi)
    return np.where(angle_rad < 0, angle_rad + 2 * np.pi, angle_rad)

angle_rad_y0 = calculate_angle(df_y0)
angle_rad_y1 = calculate_angle(df_y1)

minutes_per_bin = 15
num_bins = int(1440 / minutes_per_bin)
bin_edges = np.linspace(0, 2 * np.pi, num_bins + 1)
bin_width = 2 * np.pi / num_bins
bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2

counts_y0, _ = np.histogram(angle_rad_y0, bins=bin_edges)
counts_y1, _ = np.histogram(angle_rad_y1, bins=bin_edges)
```

```

fig, axes = plt.subplots(1, 2, figsize=(16, 8), subplot_kw={'projection': 'polar'})
plt.subplots_adjust(wspace=0.3)

hour_marks = np.arange(0, 24, 1)
hour_angles = 2 * np.pi * hour_marks / 24

hour_labels = [f'{h:02d}:00' for h in hour_marks]

ax1 = axes[0]
ax1.bar(
    bin_centers,
    counts_y1,
    width=bin_width,
    align='center',
    alpha=0.9,
    color='darkred'
)
ax1.set_title('Y=1: Sinistres Amb Morts', pad=20, fontsize=14)
ax1.set_theta_zero_location('N')
ax1.set_theta_direction(-1)
ax1.set_xticks(hour_angles)
ax1.set_xticklabels(hour_labels, fontsize=8)
ax1.set_yticklabels([])
ax1.grid(True, alpha=0.5)

ax2 = axes[1]
ax2.bar(
    bin_centers,
    counts_y0,
    width=bin_width,
    align='center',
    alpha=0.6,
    color='darkgreen'
)
ax2.set_title('Y=0: Sinistres Sense Morts', pad=20, fontsize=14)
ax2.set_theta_zero_location('N')
ax2.set_theta_direction(-1)
ax2.set_xticks(hour_angles)
ax2.set_xticklabels(hour_labels, fontsize=8)
ax2.set_yticklabels([])
ax2.grid(True, alpha=0.5)

plt.suptitle('Distribució Horària de Sinistres', fontsize=16, y=1.05)

```

```
plt.show()
```

Distribució Horària de Sinistres

